Assignment 2: ISA Simulator Implementation

Introduction

An instruction set architecture simulator is a software program that mimics how a real processor executes machine code. Our simulator reads programs written in assembly language and executes them instruction by instruction, keeping track of register values, memory contents, and the number of cycles elapsed. The processor we simulated has 16 registers (R0 to R15) 256 memory locations of data, and 256 memory locations for instructions. Each register and memory location stores an 8-bit unassigned integer, so values range from 0 to 255. Register R0 is special because it always contains 0 and cannot be changed.

The simulator was implemented in Python using object-oriented programming. Three classes were provided to us: RegisterFile for managing the 16 registers, DataMemory for storing and retrieving data values, and InstructionMemory for holding the program. Our task was to write the main simulation loop that fetched instructions, decodes them, and executes them one cycle at a time.

Architecture Overview

The simulator architecture follows the classic fetch decode execute cycle that also real processors use. At the start of each cycle, the program counter (PC) points to the address of the next instruction to execute. The simulator reads the instruction from that address, figures out what operation it represents, performs the operation, and then moves the program counter forward.

The RegisterFile class will give us methods to read and write register values. When we write a value to a register, the class automatically applies modulo 256 to keep values in the correct range. Register 10 is read only, so any attempt to write to it prints a waning and does absolutely nothing.

The DataMemory class stores the data that programs work with. Before the simulation starts, this memory is initialized from a file that specifies which addresses contain which values. During execution, the LD instruction reads from data memory into registers, and the SD instruction writes from registers into data memory. Any location that has not been explicitly initialized contains zero.

The InstructionMemory class holds the program itself. It is also initialized from a file before execution begins. Each instruction has an opcode (like ADD or JR) and up to three operands (like R1, R2, R3). The class provides methods to read the opcode and each operand separately, which makes it easy to decode instructions in our main loop.

The program counter is just a simple integer variable that we update after each instruction. For most instructions we just add one to move to the next address. For jump instructions we set the program counter to a specific value based on register contents or conditions.

Implementation Details

The core of the simulator is a while loops that runs until either the maximum number of cycles is reached or the END instruction is executed. Inside the loop we perform four steps: fetch the instruction, print status information, decode and execute the instruction, and update the program counter.

Fetching is quite straight forward, as we call the InstuctionMemory methods to read the opcode and three operands at the current program counter address.

```
    opcode = instructionMemory.read_opcode(program_counter)
    op1 = instructionMemory.read_operand_1(program_counter)
    op2 = instructionMemory.read_operand_2(program_counter)
    op3 = instructionMemory.read_operand_3(program_counter)
```

After fetching we print the cycle number, program counter value, and the instruction itself so we can follow along with what the simulator is doing.

The decode and execute phase uses a long chain of if/elif statements to check which opcode we have and perform the corresponding operation. For arithmetic instructions like ADD we read two register values, add them together, and write the result to a third register. For example.

```
    elif opcode == 'ADD': # R[op1] = R[op2] + R[op3]
        val2 = registerFile.read_register(op2)
        val3 = registerFile.read_register(op3)
        result = val2 + val3
        registerFile.write_register(op1, result)
```

The write_register method handles overflow automatically by applying modulo 256, so we do not need to worry about values going above 255.

One instruction that needed special attention was NOT. In python the bitwise NOT operator works on signed integers and gives you a negative result. For an 8-bit unsigned value we need to apply modulo 256 to get the correct answer.

```
    elif opcode == 'NOT': # % 256 Converts signed result to 8 bit unassigned
        val2 = registerFile.read_register(op2)
        result = (~val2) % 256
        registerFile.write_register(op1, result)
```

For example, NOT of 15 should give 240 but without modulo you would get negative 16.

Jump instructions were the trickiest part to be honest, because normally after executing an instruction we increase the program counter by one, but after a jump we need to skip that increment because the jump already set the program counter to the right place and we handled this with a Boolean flag which is called jumped:

```
jumped = False
```

```
elif opcode == 'JR': # unconditional jump, set PC to R[op1]
    program_counter = registerFile.read_register(op1)
    jumped = True
elif opcode == 'JEQ': # jump if equal, otherwise continue
    val2 = registerFile.read_register(op2)
    val3 = registerFile.read_register(op3)
    if val2 == val3:
        program_counter = registerFile.read_register(op1)
        jumped = True
```

```
if not jumped:
    program_counter += 1
```

This way conditional jumps only set the jumped flag if the condition is actually true. If the condition is false, jumped stays false and the program counter increments normally.
The END instruction needed to increment the cycle counter before breaking out of the loop. This is because the END instruction itself counts as a cycle.

```
if opcode == 'END': # End program
    current_cycle += 1
    break
```

Data transfer instructions like LD and SD use register values as memory addresses. For LD we read the address from a register, fetch the value at that memory location, and write it to another register. For SD we do the opposite:

```
elif opcode == 'LD': # address comes from a register value
    address = registerFile.read_register(op2)
    value = dataMemory.read_data(address)
    registerFile.write_register(op1, value)
```

At the end of the simulation, we print the final state of all registers and the data memory locations that were used during execution.

Testing

We tested the simulator with three different programs to make sure all instructions worked correctly.

Test 1

The first test program uses every instruction at least once, which loads values from memory, performs arithmetic and logic operations, stores results back to memory, and executes several different types of jumps. The program is designed so that some jump instructions are taken and others are not, which tests both branches of the conditional logic.

When we ran our simulator on Test 1, the output matched the expected results exactly. Key values were R3 equals 20 (sum of 11 and 9), R7 equals 244 (bitwise NOT of 11), R13 equals 255, R14 equals to 255 and R15 equals to 0. The fact that R15 stays at zero was important because it clearly means two LI instructions that should have been skipped by jumps were correctly not executed. The program took 24 cycles to complete, which matched the expected cycle count.

Test 2

The second test implements a bubble sort on an array of 10 numbers stored in data memory starting at address 0. The sorted result is placed in memory starting at address 10. This test is much more complex than Test 1 because it uses nested loops with conditional jumps to compare and swap values.

Our simulator produced the correct sorted array and the expected final register values and in other words confirmed that our jump logic worked properly in a realistic program with many iterations and branches.

Test 3

For the third test we wrote our own program that demonstrates several features. The program reads our four values from data memory addresses 0 through 3, adds them together in a loop and stores the sum at address 10. Then it checks whether the sum is greater than 50 and stores a flag (1 if true and 0 if false) at address 11. Finally, it performs bitwise AND, OR and NOT operations on two test values to verify that all logic instructions are working properly.

The data memory was initialized with values 15,20,8 and 12 at addresses 0 to 3 and the expected sum of them is 55 which is greater than 50 so the flag should obviously be 1. After running the simulator we get the following results:
Register values:

- R1 = 4 #final loop counter value
- R5 = 55 #the sum
- R8 = 1 #flag indicating that the sum is greater than 50
- R12 = 12 #result of 60 AND 15
- R13 = 63 #result of 60 OR 15
- R14 = 240 #result of NOT 15

Data memory values:

- Address 10 = 55 #stored sum
- Address 11 = 1 #stored flag

The simulator competed execution in 53 cycles and all values matched our manual calculations therefore confirming that the simulator handles loops, conditional logic, memory operations and bitwise operations correctly.

Conclusion

We successfully implemented a cycle accurate simulator for an 8 bit instruction set architecture. The simulator correctly executes all 13 instructions including arithmetic operations, logic operations, data transfers and control flow. Our implementation handles edge cases like overflow wrapping, the read only R0 register and the distinction between conditional and unconditional jumping.

Testing with three different programs verified that the simulator produces the correct results for individual instructions, complex loop-based algorithms and programs that combine multiple instructions types. Also the modular design using the provided RegisterFile, DataMemory and InstructionMemory classes made the implementation pretty straight forward and moderately easy to understand debugging.